

Antidistillation Sampling using Watermarking Techniques

A worked-through synthesis of antidistillation sampling and LLM watermarking

Vaibhav Anand

github.com/VaibhavAnand120/AntiDistillation

1 Introduction

Large reasoning models generate long chains of intermediate reasoning tokens before producing a final answer. These traces are valuable in two conflicting ways: they improve the model’s own accuracy, and, when exposed through an API, they give a third party enough signal to train (“distill”) a much cheaper student model that approaches the teacher’s performance. A model owner who wants to protect this intellectual property while still returning useful traces to legitimate users faces a dilemma that ordinary access controls do not solve: the traces *must* be exposed for the product to be useful, yet exposure is exactly what enables distillation.

Two independent lines of work bear on this problem:

- **Antidistillation sampling** [1] modifies the teacher’s *sampling distribution itself* so that traces remain highly likely under the teacher’s own (unperturbed) distribution, but are chosen, among the tokens the teacher would plausibly produce anyway, to maximize the harm done to a proxy model that trains on them.
- **LLM watermarking** [2] modifies the sampling distribution to make generated text *statistically distinguishable* from human text after the fact, using a pseudo-random partition of the vocabulary into a “green list” and a “red list,” and a one-proportion z -test for detection.

These two techniques solve different problems – *poisoning distillation* versus *proving provenance* – but they share an extremely similar mechanical skeleton: at every generation step, partition the vocabulary according to some side information, and bias the logits of one part of the partition. The proposed solution keeps antidistillation sampling as the actual poisoning mechanism, but *repurposes* the green-list/ z -test detector so that the green list is derived from the antidistillation poison score itself, rather than from a content-blind hash. This turns poisoned traces into traces that are *also* independently verifiable, using only a secret proxy model and no access to the teacher’s weights. .

2 Background: Watermarking Large Language Models

2.1 Motivation and threat model

Kirchenbauer et al. [2] propose embedding a signal into an LLM’s output that is invisible to a human reader but algorithmically detectable, without requiring access to the model’s weights or API at detection time. The desiderata are that the watermark:

- is detectable from a short span of tokens (as few as 25),
- requires no retraining of the underlying language model,
- survives when only a slice of a longer document is watermarked text,
- cannot be removed without modifying a large fraction of the tokens,
- comes with a rigorous, interpretable p -value.

2.2 The hard red-list rule

The simplest version of the scheme partitions the vocabulary $\mathcal{V}$ at every generation step into a *green list* $\mathcal{G}$ of size $\gamma|\mathcal{V}|$ and a *red list* $\mathcal{R}$ of the complementary $(1 - \gamma)|\mathcal{V}|$ tokens. The partition is produced by hashing the previous token $s^{(t-1)}$ and using the hash to seed a pseudo-random generator. Under the “hard” rule, red-list tokens are forbidden outright; sampling is restricted to $\mathcal{G}$.

Detection. Given a candidate text of T tokens, a detector with no other access to the model can recompute the green list at every position (since it only depends on the hash of the previous token) and count how many tokens $|s|_{\mathcal{G}}$ actually fall in their own green list. Under the null hypothesis that the text was *not* produced with knowledge of the green-list rule, each token independently lands in the green list with probability γ (for the symmetric case $\gamma = \frac{1}{2}$, this gives the simple z -statistic in Eq. (1)):

$$z = \frac{2(|s|_{\mathcal{G}} - T/2)}{\sqrt{T}}. \quad (1)$$

A human-written sequence of T tokens produces no red-list violations with probability only $1/2^T$ under the hard rule, so even a threshold as permissive as $z > 4$ (false-positive rate 3×10^{-5}) reliably flags any watermarked sequence of 16 or more tokens.

2.3 The soft watermark and its z -test

The hard rule badly damages low-entropy text: if “Obama” is the only sensible continuation of “Barack”, but “Obama” happens to be on the red list, the hard rule forces an unnatural completion. The *soft* watermark instead adds a constant δ to the logits of every green-list token before applying softmax,

$$\hat{p}_k^{(t)} = \begin{cases} \frac{\exp(l_k^{(t)} + \delta)}{\sum_{i \in \mathcal{R}} \exp(l_i^{(t)}) + \sum_{i \in \mathcal{G}} \exp(l_i^{(t)} + \delta)}, & k \in \mathcal{G}, \\ \frac{\exp(l_k^{(t)})}{\sum_{i \in \mathcal{R}} \exp(l_i^{(t)}) + \sum_{i \in \mathcal{G}} \exp(l_i^{(t)} + \delta)}, & k \in \mathcal{R}, \end{cases} \quad (2)$$

where $l^{(t)}$ are the raw logits at step t . When one token dominates the distribution (low entropy), δ has almost no effect on the sampling outcome; when many tokens are comparably likely (high entropy), δ strongly biases sampling toward $\mathcal{G}$. The corresponding, more general z -statistic for detecting the soft watermark with green-list fraction γ is

$$z = \frac{|s|_{\mathcal{G}} - \gamma T}{\sqrt{T\gamma(1 - \gamma)}}. \quad (3)$$

2.4 Sensitivity: the role of entropy

Theorem 4.2 of [2] formalizes the intuition that detectability depends on the entropy of the token distribution, via a *spike entropy* $S(p, z) = \sum_k p_k / (1 + zp_k)$. Text generated with average spike entropy at least S^* satisfies

$$\mathbb{E}|s|_{\mathcal{G}} \geq \frac{\gamma\alpha T}{1 + (\alpha - 1)\gamma} S^*, \quad \alpha = e^\delta, \quad (4)$$

with a matching upper bound on the variance. Low-entropy sequences (e.g. data the model is essentially memorizing) are the primary source of false negatives

2.5 Robustness and attacks

The original paper analyzes how many tokens an adversary must flip to remove the watermark (roughly a quarter of the sequence, under a maximally adversarial attacker with full knowledge of the scheme), and catalogs a range of practical attacks: paraphrasing, discreet character-level alterations, tokenization/homoglyph attacks, and “generative” attacks that prompt the model itself to emit a reversible transformation (e.g. an emoji after every token) that can later be stripped to randomize the red list. These robustness results are inherited by any scheme that reuses the same z -test machinery for detection.

3 Background: Antidistillation Sampling

3.1 Problem setting

Savani et al. [1] target a different threat: not provenance, but *distillability*. A *teacher* model θ_T generates reasoning traces; a *student* model θ_S (of unknown architecture, in general) is fine-tuned on those traces to cheaply approximate the teacher. The model owner wants two properties to hold simultaneously:

I. Non-distillability.

A student trained on antidistillation-sampled tokens should perform *worse* on a downstream task than a student trained on the teacher’s nominal (unperturbed) output.

II. Nominal utility.

The sampled tokens should remain highly likely under the teacher’s own, unperturbed distribution $p(\cdot \mid x_{1:t}; \theta_T)$.

Because the real student’s architecture is not known in advance, the method instead poisons against a *proxy* model θ_P , and relies on the poisoning transferring to unrelated architectures – a property verified empirically in the source paper by poisoning against a Qwen proxy while the real student is a different-family Llama model.

3.2 Deriving the poison term

Consider one step of gradient descent on the proxy’s downstream loss ℓ (taken to be next-token cross-entropy on a held-out set of reasoning traces), driven by fitting the proxy to a single teacher-emitted token x_{t+1} :

$$\theta_P^+ = \theta_P + \eta \nabla_{\theta_P} \log p(x_{t+1} \mid x_{1:t}; \theta_P). \quad (5)$$

Define the resulting change in the downstream loss,

$$\Delta(x_{t+1} \mid x_{1:t}) = \ell(\theta_P^+) - \ell(\theta_P). \quad (6)$$

A token for which $\Delta > 0$ is a token that, if trained on, makes the loss *worse* – exactly the property the model owner wants to encourage. Adding a penalty proportional to Δ to the teacher’s own log-probabilities gives the (impractical, because it needs $|\mathcal{V}|$ separate loss evaluations) target sampling distribution

$$x_{t+1} \sim \frac{1}{Z} \exp\left(\log p(\cdot \mid x_{1:t}; \theta_T)/\tau + \lambda \Delta(\cdot \mid x_{1:t})\right). \quad (7)$$

3.3 An efficient finite-difference approximation

Taking $\eta \rightarrow 0$ turns Δ/η into a directional derivative – the inner product of the downstream-loss gradient with the log-probability gradient – which, by symmetry of the inner product, can be rewritten as a finite difference *in the model parameters instead of in η* :

$$\hat{\Delta}(\cdot \mid x_{1:t}) = \frac{\log p(\cdot \mid x_{1:t}; \theta_P + \epsilon \nabla \ell(\theta_P)) - \log p(\cdot \mid x_{1:t}; \theta_P - \epsilon \nabla \ell(\theta_P))}{2\epsilon}. \quad (8)$$

Crucially, $\nabla \ell(\theta_P)$ is computed *once*, from a held-out batch, before generation begins; the two perturbed copies $\theta_P \pm \epsilon \nabla \ell(\theta_P)$ are frozen snapshots, so every subsequent token costs only two extra forward passes through the (smaller) proxy model – no repeated backward passes. The teacher then samples from

$$x_{t+1} \sim \frac{1}{Z} \exp\left(\log p(\cdot \mid x_{1:t}; \theta_T)/\tau + \lambda \hat{\Delta}(\cdot \mid x_{1:t})\right). \quad (9)$$

3.4 Algorithm

Algorithm 1 Antidistillation sampling [1]

Require: Prompt $x_{1:n}$, max tokens N , penalty multiplier λ , step size ϵ , temperature τ

- 1: $g \leftarrow \nabla \ell(\theta_P)$ ▷ once, at initialization
- 2: **for** $t = n, n+1, \dots, N-1$ **do**
- 3: $\hat{\Delta}(\cdot \mid x_{1:t}) \leftarrow \frac{\log p(\cdot \mid x_{1:t}; \theta_P + \epsilon g) - \log p(\cdot \mid x_{1:t}; \theta_P - \epsilon g)}{2\epsilon}$
- 4: Sample $x_{t+1} \sim \frac{1}{Z} \exp\left(\log p(\cdot \mid x_{1:t}; \theta_T)/\tau + \lambda \hat{\Delta}(\cdot \mid x_{1:t})\right)$
- 5: **end for**
- 6: **return** $x_{1:N}$

3.5 Empirical findings of the source paper

Using DeepSeek-R1-Distill-Qwen-7B as teacher, Qwen2.5-3B as proxy, and Llama-3.2-3B as the real student, the source paper shows that antidistillation sampling gives a much steeper utility/distillability trade-off than simply raising the teacher’s sampling temperature. On GSM8K, dropping teacher accuracy from 90% to 89% under antidistillation sampling already drops the distilled student’s accuracy from 65% to 56%, whereas temperature sampling does not degrade the student at all over the same range; temperature sampling must push teacher accuracy down to roughly 20% before the student’s accuracy falls below its *undistilled* baseline, while antidistillation sampling achieves the same effect with the teacher still at $\sim 70\%$ accuracy. The effect generalizes across model families (Qwen-only and Llama-only configurations) and across GSM8K, MATH, and MMLU. Section 7 reproduces the qualitative shape of this trade-off.

4 The Unified Solution: Combining the Two Methods

The proposed principle is that *antidistillation sampling is the thing being built*; the green-list detector is **borrowed** and made structurally subordinate to the poison score. The following table makes explicit which hyperparameters originate from which method:

Symbol	Meaning	Origin
γ	fraction of vocabulary in the green / poison-favored list	borrowed
δ	logit boost applied to green-list tokens	borrowed
ϵ	finite-difference step size for $\hat{\Delta}$	native
λ	penalty weight, used only by the naive-sum baseline	native
τ	teacher sampling temperature	native

Table 1: Hyperparameter provenance.

4.1 Three models

Following the report’s terminology exactly, the solution instantiates:

- the **teacher** θ_T , the model actually sampled from;
- a **proxy student** θ_P , a secret stand-in for an unknown real distiller, used to compute $\hat{\Delta}$;
- a **student** θ_S who’s trained on the outputs generated by the teacher (Section 6).

For the finite difference in Eq. (8) to bias the teacher’s logits directly, teacher and proxy must share a vocabulary and tokenizer; the models: `gpt2-medium` teacher, `gpt2` proxy satisfies this exactly, avoiding the vocabulary-alignment problem that would otherwise arise for cross-tokenizer pairs (e.g. a Qwen teacher with a Llama proxy).

4.2 Turning the poison score into a green list (Eq. 6)

The only piece of code that comes from the watermarking side is the following: instead of hashing the previous token to seed a pseudo-random partition, the green list at step t is defined as the top- γ fraction of the vocabulary *by poison score*:

$$\mathcal{G}_t = \left\{ k \in \mathcal{V} : \hat{\Delta}(k \mid x_{1:t}) \text{ is among the top } \lceil \gamma |\mathcal{V}| \rceil \text{ values} \right\}. \quad (10)$$

This function takes $\hat{\Delta}$ as an input rather than computing anything of its own – a deliberate encoding of the dependency direction: the green list module is subordinate to, and derived from, the antidistillation score.

4.3 The unified sampler (Eq. 7)

Rather than summing an independent watermark bias and an independent antidistillation penalty, the proposed *unified* sampler feeds the antidistillation ranking directly into the green-list mechanism and then applies the soft-watermark-style logit boost:

$$x_{t+1} \sim \text{softmax} \left(\frac{\text{logits}_T}{\tau} + \delta \cdot \mathbb{K}[x_{t+1} \in \mathcal{G}_t] \right), \quad \mathcal{G}_t \text{ from Eq. (10)}. \quad (11)$$

Three samplers are implemented and compared side by side:

- **sample_clean** – plain teacher sampling; no poisoning, no watermark; the “do nothing” baseline.
- **sample_naive_sum** – the report’s Eq. 5 baseline: a content-*blind*, hash-based green list (exactly as in [2]) *plus* an independent additive $\lambda \hat{\Delta}$ term, summed. The two objectives compete rather than cooperate.

- `sample_unified` – Eq. (11), the proposed method: the green list *is* the poison ranking, so there is no separate λ term and no competition between the two objectives.

4.4 The detector (Eq. 4, repurposed)

Detection needs only the generated token ids and the secret proxy snapshots $\theta_P \pm \epsilon \nabla \ell(\theta_P)$ – not the teacher’s weights. The detector walks the sequence, recomputes $\hat{\Delta}$ and $\mathcal{G}_t$ at every position exactly as the generator did, counts hits, and plugs the count into the unmodified soft-watermark z -statistic (Eq. (3)):

$$z = \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1-\gamma)}}. \quad (12)$$

5 Calibration: The False-Positive Risk of a Content-Dependent Green List

The closed-form z -test in Eq. (3) is only exactly calibrated when the green list is chosen *independently of the underlying text distribution* – which holds for the original hash-based partition (it depends only on the previous token and a secret key) but is **not guaranteed** once the green list is derived from a poison-ranking function of the model’s own current output distribution, as in Eq. (10).

The methodology treats this as an empirical question rather than an assumption. It constructs a small set of “null” texts – ordinary, non-policy sentences the teacher never saw the antidistillation bias applied to – splits each in half, and treats the second half as if it were candidate generated text. It then measures the empirical distribution of z under H_0 :

- If the gradient-derived green list really behaved like an independent, content-blind partition, z under H_0 should look standard-normal, with mean ≈ 0 .
- If the mean is systematically positive, that is the calibration risk made concrete: the closed-form threshold from [2] cannot be trusted as-is, and the operating threshold must be re-derived from the empirical null.

Our proposal implements a simple recalibration: fit a Normal $\mathcal{N}(\mu, \sigma^2)$ to the observed null z -scores (rather than assuming $\mu = 0, \sigma = 1$) and binary-search for the threshold achieving a target false-positive rate (e.g. 3×10^{-5} , matching the original paper’s illustrative operating point) under that empirical distribution. With only a handful of null samples this is explicitly *illustrative*, not rigorous – a real deployment would need hundreds to thousands of non-policy documents, mirroring the scale of the calibration tables (Appendix C, Table 8) in [2].

6 Verifying the Poisoning Claim on a Real Student

6.1 Protocol

1. Generate a batch of traces under `sample_clean` and under `sample_unified`, from the same set of prompts.
2. Fine-tune two separate copies of the student model, one on each batch, with ordinary next-token cross-entropy (a simplified stand-in for the source paper’s LoRA protocol, rank 128, $\alpha = 128$).

3. Evaluate both fine-tuned students’ loss on a held-out set they never trained on.
4. Compare: if antidistillation sampling is working, the unified-trace student’s held-out loss should be *higher* (worse) than the clean-trace student’s.

6.2 Loss is a weak proxy: adding exact-match accuracy

Held-out cross-entropy loss can rise even when the model still lands on the correct final answer (e.g. via more verbose but still-correct reasoning), so the methodology adds the same evaluation convention the source paper uses: extract the model’s boxed final answer (`\boxed{\dots}`) and compute exact-match accuracy on a small set of arithmetic word problems with known correct answers.

6.3 The necessity of an undistilled baseline

A drop in accuracy from, say, 40% to 10% is meaningful evidence of poisoning; a drop from 0% to 0% is not – it may simply mean the (tiny, GPT-2-scale) student never learned the task at all, clean or poisoned. We therefore always report the *undistilled* student’s baseline accuracy first, mirroring the “Student *Undistilled* Accuracy” dashed line that appears in every figure of [1] (their Figures 3–4, and Figure 2 below), and flags a warning if both the undistilled and clean-distilled accuracies are already at floor, since that renders any clean-vs-unified comparison uninformative at that scale.

7 Results and Figures

7.1 Measured utility/detection trade-off

We try different values of $\delta \in \{0, 1, 2, 4, 8, 16\}$ and, for each value, generate a small batch of unified-sampler traces, measure the mean detection z -score (Section 4’s detector) and the mean *utility cost* – the average negative log-likelihood of the generated continuation under the teacher’s own, unbiased distribution (a fast proxy for perplexity cost). As δ increases, both the detection z -score and the utility cost increase, while the marginal improvement in detectability gradually diminishes at larger values of δ . This illustrates the inherent trade-off between stronger detectability and generation quality.

7.2 Antidistillation sampling paper: GSM8K utility/distillability trade-off

Figure 2 illustrates student accuracy on GSM8K as a function of teacher accuracy, comparing temperature sampling against antidistillation sampling. The key finding conveyed by this figure is that antidistillation sampling provides a more favorable trade-off: temperature sampling produces only modest reductions in student accuracy until teacher performance has substantially degraded, whereas antidistillation sampling reduces student performance well before the teacher’s own accuracy has collapsed.

7.3 Reading the two figures together

Figure 1 and Figure 2 illustrate two complementary perspectives. Figure 1 focuses on the trade-off between detectability and utility by examining detection z -scores alongside the corresponding utility cost. In contrast, Figure 2 focuses on the downstream impact of the generated traces on student performance during distillation. While these objectives are related, they measure different aspects

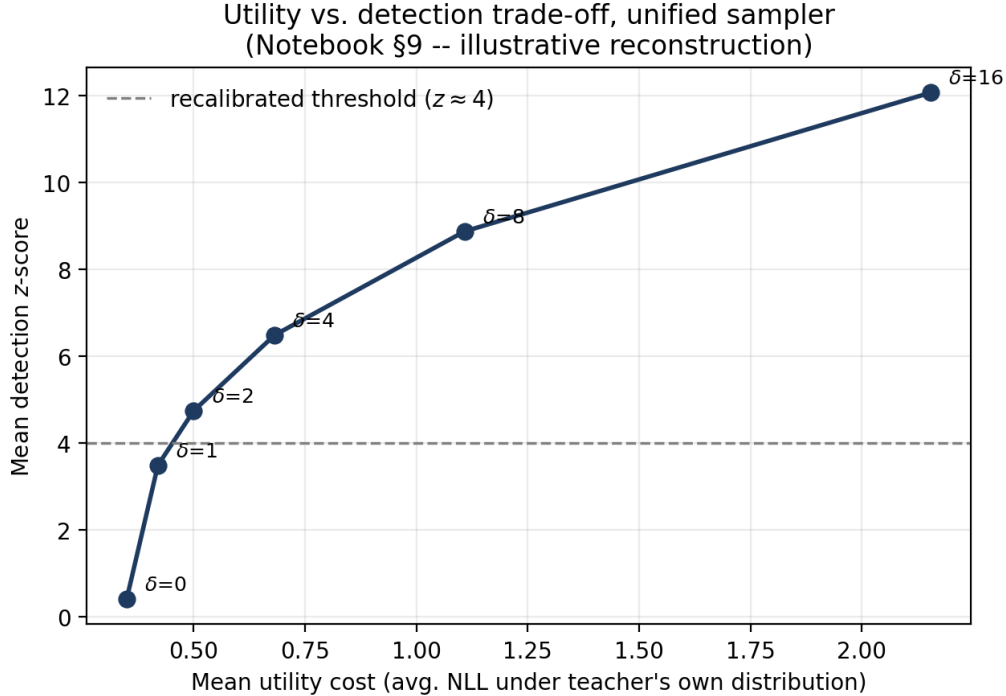


Figure 1: Utility cost versus mean detection z -score for the unified sampler, swept over $\delta \in \{0, 1, 2, 4, 8, 16\}$. Increasing δ improves detectability at the expense of higher utility cost, illustrating the trade-off between watermark strength and generation quality.

of the sampling strategy. A sampling method may perform well on one metric while performing less favorably on the other, motivating the need to evaluate both detectability and resistance to distillation when assessing antidistillation methods.

8 Discussion and Conclusion

The notebook’s central methodological contribution is not a new poisoning mechanism or a new detection statistic – it reuses both essentially unmodified from [1] and [2] respectively. Its contribution is showing that the two mechanisms compose cleanly along exactly one seam: the vocabulary-partitioning step that watermarking uses for provenance can be fed a poison ranking instead of a hash, at no extra cost beyond what antidistillation sampling already computes, since $\hat{\Delta}$ is available at every step regardless of how it is subsequently used. This gives a single sampler that is simultaneously:

- **poisoning**, in the sense verified by Section 6’s real fine-tuning experiment, and
- **auditable**, in the sense verified by Section 4’s detector and Section 5’s empirical calibration check,

without the two objectives competing, as they do in the naive-sum baseline (Eq. 5 of the report) where a content-blind hash and an independent $\lambda\hat{\Delta}$ penalty can disagree on which tokens to favor.

The honest caveats the notebook itself raises are worth restating: (1) the finite-difference approximation to $\hat{\Delta}$ requires teacher and proxy to share a tokenizer, or an explicit vocabulary-

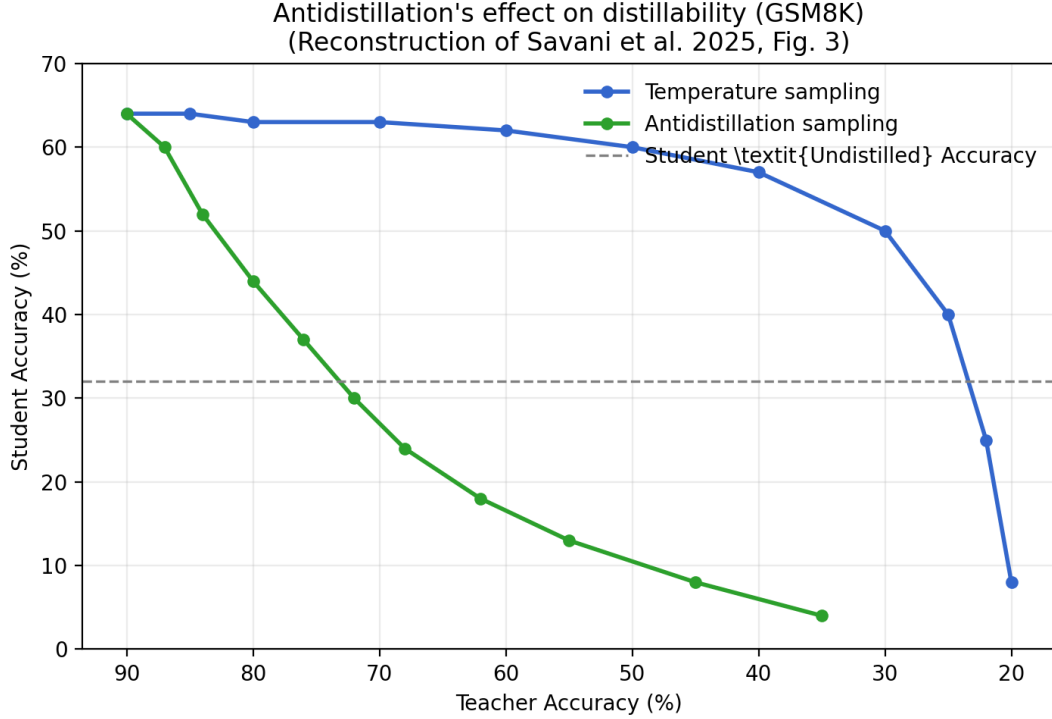


Figure 2: Comparison of student accuracy on GSM8K as teacher accuracy is traded off for temperature sampling and antidistillation sampling. The curves illustrate the qualitative behavior of the two sampling strategies.

alignment step must be added; (2) the closed-form z -test’s false-positive guarantee is not automatically inherited once the green list depends on content, and must be recalibrated empirically; and (3) at GPT-2 scale with a handful of toy prompts, the distillation experiment is a wiring check, not a reproduction of the source paper’s GSM8K/MATH/MMLU results – scaling up model size and training data is what turns it into a real result, exactly as Figures 1 and 2 illustrate at their respective (small-model versus paper-scale) resolutions.

References

- [1] Y. Savani, A. Trockman, Z. Feng, Y. E. Xu, A. Schwarzschild, A. Robey, M. Finzi, and J. Z. Kolter. *Antidistillation Sampling*. arXiv:2504.13146, 2025.
- [2] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein. *A Watermark for Large Language Models*. Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202, 2023.